SREEJA REDDY YELURU

Exploring Word Vectors

This code is adopted almost verbatim from Assign #1 from Stanford University, CS224n Natural Language Processing with Deep Learning course (https://web.stanford.edu/class/cs224n/).

==========

Before you start, make sure you **read the README.md** in the same directory as this notebook for important setup information. You need to install some Python libraries before you can successfully do this assignment. A lot of code is provided in this notebook, and we highly encourage you to read and understand it as part of the learning :)

If you aren't super familiar with Python, Numpy, or Matplotlib, the CS231N Python/Numpy tutorial is a great resource.

```
!pip install gensim
!pip install datasets
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.26.4)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.13.1)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.3.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (1.17.3
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.19.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.26.4)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.5.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.35.
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-h
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dat
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->dat
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas-
```

```
# All Import Statements Defined Here
# Note: Do not add to this list.
```

```
# ----------------

import sys
assert sys.version_info[0] == 3
assert sys.version_info[1] >= 8

from platform import python_version
assert int(python_version().split(".")[1]) >= 5, "Please upgrade your Python version following the instructions in \
    the README.md file found in the same directory as this notebook. Your Python version is " + python_version()

from gensim.models import KeyedVectors
from gensim.test.utils import datapath
import pprint
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [10, 5]

from datasets import load_dataset
imdb_dataset = load_dataset("stanfordnlp/imdb", name="plain_text")

import re
import numpy as np
import random
import scipy as sp
from sklearn.decomposition import TruncatedSVD
from sklearn.decomposition import PCA

START_TOKEN = '<START>'
END_TOKEN = '<END>'
NUM_SAMPLES = 150

np.random.seed(0)
random.seed(0)
# ----------------
```

Word Vectors

Word Vectors are often used as a fundamental component for downstream NLP tasks, e.g. question answering, text generation, translation, etc., so it is important to build some intuitions as to their strengths and weaknesses. Here, you will explore two types of word vectors: those derived from *co-occurrence matrices*, and those derived via *GloVe*.

Note on Terminology: The terms "word vectors" and "word embeddings" are often used interchangeably. The term "embedding" refers to the fact that we are encoding aspects of a word's meaning in a lower dimensional space. As Wikipedia states, "*conceptually it involves a mathematical embedding from a space with one dimension per word to a continuous vector space with a much lower dimension*".

Part 1: Count-Based Word Vectors

Most word vector models start from the following idea:

You shall know a word by the company it keeps (Firth, J. R. 1957:11)

Many word vector implementations are driven by the idea that similar words, i.e., (near) synonyms, will be used in similar contexts. As a result, similar words will often be spoken or written along with a shared subset of words, i.e., contexts. By examining these contexts, we can try to develop embeddings for our words. With this intuition in mind, many "old school" approaches to constructing word vectors relied on word counts. Here we elaborate upon one of those strategies, *co-occurrence matrices* (for more information, see here

or here).

Co-Occurrence

A co-occurrence matrix counts how often things co-occur in some environment. Given some word w_i occurring in the document, we consider the *context window* surrounding w_i. Supposing our fixed window size is n, then this is the n **preceding and** n **subsequent words** in that document, i.e. words $w_{i-n} \dots w_{i-1}$ and $w_{i+1} \dots w_{i+n}$. We build a *co-occurrence matrix* M, which is a symmetric word-by-word matrix in which M_{ij} is the number of times w_j appears inside w_i's window among all documents.

Example: Co-Occurrence with Fixed Window of n=1:

Document 1: "all that glitters is not gold"

Document 2: "all is well that ends well"

*	<START>	all	that	glitters	is	not	gold	well	ends	<END>
<START>	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
<END>	0	0	0	0	0	0	1	1	0	0

In NLP, we commonly use `<START>` and `<END>` tokens to mark the beginning and end of sentences, paragraphs, or documents. These tokens are included in co-occurrence counts, encapsulating each document, for example: "`<START>` All that glitters is not gold `<END>`".

The matrix rows (or columns) provide word vectors based on word-word co-occurrence, but they can be large. To reduce dimensionality, we employ **Singular Value Decomposition (SVD)**, akin to PCA, selecting the top k principal components. The SVD process decomposes the co-occurrence matrix A into singular values in the diagonal S matrix and new, shorter word vectors in U_k.

This dimensionality reduction maintains semantic relationships; for instance, *doctor* and *hospital* will be closer than *doctor* and *dog*.

Plotting Co-Occurrence Word Embeddings

Here, we will be using the Large Movie Review Dataset. This is a dataset for binary sentiment classification containing substantially more data than previous benchmark datasets. We provide a set of 25,000 highly polar movie reviews for training, and 25,000 for testing. There is additional unlabeled data for use as well. We provide a `read_corpus` function below that pulls out the text of a movie review from the dataset. The function also adds `<START>` and `<END>` tokens to each of the documents, and lowercases words. You do **not** have to perform any other kind of pre-processing.

```
def read_corpus():
    """ Read files from the Large Movie Review Dataset.
        Params:
            category (string): category name
        Return:
            list of lists, with words from each of the processed files
    """
    files = imdb_dataset["train"]["text"][:NUM_SAMPLES]
    return [[START_TOKEN] + [re.sub(r'[^\w]', '', w.lower()) for w in f.split(" ")] + [END_TOKEN] for f in files]
```

Let's have a look what these documents are like....

```
imdb_corpus = read_corpus()
pprint.pprint(imdb_corpus[:3], compact=True, width=100)
print("corpus size: ", len(imdb_corpus[0]))
```

```
[['<START>', 'i', 'rented', 'i', 'am', 'curiousyellow', 'from', 'my', 'video', 'store', 'because',
  'of', 'all', 'the', 'controversy', 'that', 'surrounded', 'it', 'when', 'it', 'was', 'first',
  'released', 'in', '1967', 'i', 'also', 'heard', 'that', 'at', 'first', 'it', 'was', 'seized',
  'by', 'us', 'customs', 'if', 'it', 'ever', 'tried', 'to', 'enter', 'this', 'country', 'therefore',
  'being', 'a', 'fan', 'of', 'films', 'considered', 'controversial', 'i', 'really', 'had', 'to',
  'see', 'this', 'for', 'myselfbr', 'br', 'the', 'plot', 'is', 'centered', 'around', 'a', 'young',
  'swedish', 'drama', 'student', 'named', 'lena', 'who', 'wants', 'to', 'learn', 'everything',
  'she', 'can', 'about', 'life', 'in', 'particular', 'she', 'wants', 'to', 'focus', 'her',
  'attentions', 'to', 'making', 'some', 'sort', 'of', 'documentary', 'on', 'what', 'the', 'average',
  'swede', 'thought', 'about', 'certain', 'political', 'issues', 'such', 'as', 'the', 'vietnam',
  'war', 'and', 'race', 'issues', 'in', 'the', 'united', 'states', 'in', 'between', 'asking',
  'politicians', 'and', 'ordinary', 'denizens', 'of', 'stockholm', 'about', 'their', 'opinions',
  'on', 'politics', 'she', 'has', 'sex', 'with', 'her', 'drama', 'teacher', 'classmates', 'and',
  'married', 'menbr', 'br', 'what', 'kills', 'me', 'about', 'i', 'am', 'curiousyellow', 'is',
  'that', '40', 'years', 'ago', 'this', 'was', 'considered', 'pornographic', 'really', 'the', 'sex',
  'and', 'nudity', 'scenes', 'are', 'few', 'and', 'far', 'between', 'even', 'then', 'its', 'not',
  'shot', 'like', 'some', 'cheaply', 'made', 'porno', 'while', 'my', 'countrymen', 'mind', 'find',
  'it', 'shocking', 'in', 'reality', 'sex', 'and', 'nudity', 'are', 'a', 'major', 'staple', 'in',
  'swedish', 'cinema', 'even', 'ingmar', 'bergman', 'arguably', 'their', 'answer', 'to', 'good',
  'old', 'boy', 'john', 'ford', 'had', 'sex', 'scenes', 'in', 'his', 'filmsbr', 'br', 'i', 'do',
```

```
 'commend', 'the', 'filmmakers', 'for', 'the', 'fact', 'that', 'any', 'sex', 'shown', 'in', 'the',
 'film', 'is', 'shown', 'for', 'artistic', 'purposes', 'rather', 'than', 'just', 'to', 'shock',
 'people', 'and', 'make', 'money', 'to', 'be', 'shown', 'in', 'pornographic', 'theaters', 'in',
 'america', 'i', 'am', 'curiousyellow', 'is', 'a', 'good', 'film', 'for', 'anyone', 'wanting',
 'to', 'study', 'the', 'meat', 'and', 'potatoes', 'no', 'pun', 'intended', 'of', 'swedish',
 'cinema', 'but', 'really', 'this', 'film', 'doesnt', 'have', 'much', 'of', 'a', 'plot', '<END>'],
['<START>', 'i', 'am', 'curious', 'yellow', 'is', 'a', 'risible', 'and', 'pretentious', 'steaming',
 'pile', 'it', 'doesnt', 'matter', 'what', 'ones', 'political', 'views', 'are', 'because', 'this',
 'film', 'can', 'hardly', 'be', 'taken', 'seriously', 'on', 'any', 'level', 'as', 'for', 'the',
 'claim', 'that', 'frontal', 'male', 'nudity', 'is', 'an', 'automatic', 'nc17', 'that', 'isnt',
 'true', 'ive', 'seen', 'rrated', 'films', 'with', 'male', 'nudity', 'granted', 'they', 'only',
 'offer', 'some', 'fleeting', 'views', 'but', 'where', 'are', 'the', 'rrated', 'films', 'with',
 'gaping', 'vulvas', 'and', 'flapping', 'labia', 'nowhere', 'because', 'they', 'dont', 'exist',
 'the', 'same', 'goes', 'for', 'those', 'crappy', 'cable', 'shows', 'schlongs', 'swinging', 'in',
 'the', 'breeze', 'but', 'not', 'a', 'clitoris', 'in', 'sight', 'and', 'those', 'pretentious',
 'indie', 'movies', 'like', 'the', 'brown', 'bunny', 'in', 'which', 'were', 'treated', 'to', 'the',
 'site', 'of', 'vincent', 'gallos', 'throbbing', 'johnson', 'but', 'not', 'a', 'trace', 'of',
 'pink', 'visible', 'on', 'chloe', 'sevigny', 'before', 'crying', 'or', 'implying',
 'doublestandard', 'in', 'matters', 'of', 'nudity', 'the', 'mentally', 'obtuse', 'should', 'take',
 'into', 'account', 'one', 'unavoidably', 'obvious', 'anatomical', 'difference', 'between', 'men',
 'and', 'women', 'there', 'are', 'no', 'genitals', 'on', 'display', 'when', 'actresses', 'appears',
 'nude', 'and', 'the', 'same', 'cannot', 'be', 'said', 'for', 'a', 'man', 'in', 'fact', 'you',
 'generally', 'wont', 'see', 'female', 'genitals', 'in', 'an', 'american', 'film', 'in',
 'anything', 'short', 'of', 'porn', 'or', 'explicit', 'erotica', 'this', 'alleged',
 'doublestandard', 'is', 'less', 'a', 'double', 'standard', 'than', 'an', 'admittedly',
 'depressing', 'ability', 'to', 'come', 'to', 'terms', 'culturally', 'with', 'the', 'insides',
 'of', 'womens', 'bodies', '<END>'],
['<START>', 'if', 'only', 'to', 'avoid', 'making', 'this', 'type', 'of', 'film', 'in', 'the',
 'future', 'this', 'film', 'is', 'interesting', 'as', 'an', 'experiment', 'but', 'tells', 'no',
 'cogent', 'storybr', 'br', 'one', 'might', 'feel', 'virtuous', 'for', 'sitting', 'thru', 'it',
 'because', 'it', 'touches', 'on', 'so', 'many', 'important', 'issues', 'but', 'it', 'does', 'so',
 'without', 'any', 'discernable', 'motive', 'the', 'viewer', 'comes', 'away', 'with', 'no', 'new',
 'perspectives', 'unless', 'one', 'comes', 'up', 'with', 'one', 'while', 'ones', 'mind', 'wanders',
 'as', 'it', 'will', 'invariably', 'do', 'during', 'this', 'pointless', 'filmbr', 'br', 'one',
 'might', 'better', 'spend', 'ones', 'time', 'staring', 'out', 'a', 'window', 'at', 'a', 'tree',
 'growingbr', 'br', '', '<END>']]
corpus size:  290
```

Implement `distinct_words`

Write a method to work out the distinct words (word types) that occur in the corpus.

You can use `for` loops to process the input `corpus` (a list of list of strings), but try using Python list comprehensions (which are generally faster). In particular, this may be useful to flatten a list of lists. If you're not familiar with Python list comprehensions in general, here's more information.

Your returned `corpus_words` should be sorted. You can use python's `sorted` function for this.

You may find it useful to use Python sets to remove duplicate words.

```
def distinct_words(corpus):
    """ Determine a list of distinct words for the corpus.
        Params:
            corpus (list of list of strings): corpus of documents
        Return:
            corpus_words (list of strings): sorted list of distinct words across the corpus
            n_corpus_words (integer): number of distinct words across the corpus
    """
    corpus_words = []
    n_corpus_words = -1

    # --------------------
    # Write your implementation here.

    all_unique_words = set() # collecting all unique words
    for each_document in corpus: # iterating through each one inside the corpus and adding into set to remove duplicate
        for each_word in each_document:
            all_unique_words.add(each_word)
    corpus_words = sorted(list(all_unique_words)) # converting set into a sorted list
    n_corpus_words = len(corpus_words)   # calculating total no of distinct words

    # --------------------

    return corpus_words, n_corpus_words
```

```
# ---------------------
# Run this sanity check
```

```
# Note that this not an exhaustive check for correctness.
# ----------------------

# Define toy corpus
test_corpus = ["{} All that glitters isn't gold {}".format(START_TOKEN, END_TOKEN).split(" "), "{} All's well that ends
test_corpus_words, num_corpus_words = distinct_words(test_corpus)

# Correct answers
ans_test_corpus_words = sorted([START_TOKEN, "All", "ends", "that", "gold", "All's", "glitters", "isn't", "well", END_T
ans_num_corpus_words = len(ans_test_corpus_words)

# Test correct number of words
assert(num_corpus_words == ans_num_corpus_words), "Incorrect number of distinct words. Correct: {}. Yours: {}".format(a

# Test correct words
assert (test_corpus_words == ans_test_corpus_words), "Incorrect corpus_words.\nCorrect: {}\nYours:   {}".format(str(ans

# Print Success
print ("-" * 80)
print("Passed All Tests!")
print ("-" * 80)
```

```
--------------------------------------------------------------------------------
Passed All Tests!
--------------------------------------------------------------------------------
```

Implement `compute_co_occurrence_matrix`

Write a method that constructs a co-occurrence matrix for a certain window-size n (with a default of 4), considering words n before and n after the word in the center of the window. Here, we start to use `numpy (np)` to represent vectors, matrices, and tensors.

```
def compute_co_occurrence_matrix(corpus, window_size=4):
    """ Compute co-occurrence matrix for the given corpus and window_size (default of 4).

        Note: Each word in a document should be at the center of a window. Words near edges will have a smaller
              number of co-occurring words.

              For example, if we take the document "<START> All that glitters is not gold <END>" with window size of 4,
              "All" will co-occur with "<START>", "that", "glitters", "is", and "not".

        Params:
            corpus (list of list of strings): corpus of documents
            window_size (int): size of context window
        Return:
            M (a symmetric numpy matrix of shape (number of unique words in the corpus , number of unique words in the
                Co-occurence matrix of word counts.
                The ordering of the words in the rows/columns should be the same as the ordering of the words given by
            word2ind (dict): dictionary that maps word to index (i.e. row/column number) for matrix M.
    """
    words, n_words = distinct_words(corpus)
    M = None
    word2ind = {}

    # ------------------
    # Write your implementation here.

    for each_index in range(n_words): # creating word2ind dictionary by assigning index for every word
        word2ind[words[each_index]] = each_index
    M = np.zeros((n_words, n_words)) # initializing the co-occurrence matrix with zeros
    for each_document in corpus: # going through each one in corpus and iterating word by word
        for current_index in range(len(each_document)):
            current_word = each_document[current_index]
            current_word_index = word2ind[current_word]
            start_index = max(0, current_index - window_size)  # calculating context window range
            end_index = min(len(each_document), current_index + window_size + 1)
            for neighbor_index in range(start_index, end_index): # looping through context window
                if neighbor_index != current_index:
                    context_word = each_document[neighbor_index]
                    context_word_index = word2ind[context_word]
                    M[current_word_index][context_word_index] += 1 # increasing count
    # ------------------

    return M, word2ind
```

```
# ---------------------
# Run this sanity check
# Note that this is not an exhaustive check for correctness.
# ---------------------

# Define toy corpus and get student's co-occurrence matrix
test_corpus = ["{} All that glitters isn't gold {}".format(START_TOKEN, END_TOKEN).split(" "), "{} All's well that ends
M_test, word2ind_test = compute_co_occurrence_matrix(test_corpus, window_size=1)

# Correct M and word2ind
M_test_ans = np.array(
    [[0., 0., 0., 0., 0., 0., 1., 0., 0., 1.,],
     [0., 0., 1., 1., 0., 0., 0., 0., 0., 0.,],
     [0., 1., 0., 0., 0., 0., 0., 0., 1., 0.,],
     [0., 1., 0., 0., 0., 0., 0., 0., 0., 1.,],
     [0., 0., 0., 0., 0., 0., 0., 0., 1., 1.,],
     [0., 0., 0., 0., 0., 0., 0., 1., 1., 0.,],
     [1., 0., 0., 0., 0., 0., 0., 1., 0., 0.,],
     [0., 0., 0., 0., 0., 1., 1., 0., 0., 0.,],
     [0., 0., 1., 0., 1., 1., 0., 0., 0., 1.,],
     [1., 0., 0., 1., 1., 0., 0., 0., 1., 0.,]]
)
ans_test_corpus_words = sorted([START_TOKEN, "All", "ends", "that", "gold", "All's", "glitters", "isn't", "well", END_T
word2ind_ans = dict(zip(ans_test_corpus_words, range(len(ans_test_corpus_words))))

# Test correct word2ind
assert (word2ind_ans == word2ind_test), "Your word2ind is incorrect:\nCorrect: {}\nYours: {}".format(word2ind_ans, word

# Test correct M shape
assert (M_test.shape == M_test_ans.shape), "M matrix has incorrect shape.\nCorrect: {}\nYours: {}".format(M_test.shape,

# Test correct M values
for w1 in word2ind_ans.keys():
    idx1 = word2ind_ans[w1]
    for w2 in word2ind_ans.keys():
        idx2 = word2ind_ans[w2]
        student = M_test[idx1, idx2]
        correct = M_test_ans[idx1, idx2]
        if student != correct:
            print("Correct M:")
            print(M_test_ans)
            print("Your M: ")
            print(M_test)
            raise AssertionError("Incorrect count at index ({}, {})=({}, {}) in matrix M. Yours has {} but should have

# Print Success
print ("-" * 80)
print("Passed All Tests!")
print ("-" * 80)
```

```
--------------------------------------------------------------------------------
Passed All Tests!
--------------------------------------------------------------------------------
```

Implement `reduce_to_k_dim`

Construct a method that performs dimensionality reduction on the matrix to produce k-dimensional embeddings. Use SVD to take the top k components and produce a new matrix of k-dimensional embeddings.

Note: All of numpy, scipy, and scikit-learn (`sklearn`) provide *some* implementation of SVD, but only scipy and sklearn provide an implementation of Truncated SVD, and only sklearn provides an efficient randomized algorithm for calculating large-scale Truncated SVD. So please use sklearn.decomposition.TruncatedSVD.

```
def reduce_to_k_dim(M, k=2):
    """ Reduce a co-occurence count matrix of dimensionality (num_corpus_words, num_corpus_words)
        to a matrix of dimensionality (num_corpus_words, k) using the following SVD function from Scikit-Learn:
            - http://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html

        Params:
            M (numpy matrix of shape (number of unique words in the corpus , number of unique words in the corpus)): co
            k (int): embedding size of each word after dimension reduction
        Return:
            M_reduced (numpy matrix of shape (number of corpus words, k)): matrix of k-dimensioal word embeddings.
                    In terms of the SVD from math class, this actually returns U * S
```

```
    """
    n_iters = 10    # Use this parameter in your call to `TruncatedSVD`
    M_reduced = None
    print("Running Truncated SVD over %i words..." % (M.shape[0]))

    # ------------------
    # Write your implementation here.
    svd = TruncatedSVD(n_components=k, n_iter=n_iters) # initialising truncated svd
    M_reduced = svd.fit_transform(M) # applying svd on matrix
    # ------------------

    print("Done.")
    return M_reduced
```

```
# ---------------------
# Run this sanity check
# Note that this is not an exhaustive check for correctness
# In fact we only check that your M_reduced has the right dimensions.
# ---------------------

# Define toy corpus and run student code
test_corpus = ["{} All that glitters isn't gold {}".format(START_TOKEN, END_TOKEN).split(" "), "{} All's well that ends
M_test, word2ind_test = compute_co_occurrence_matrix(test_corpus, window_size=1)
M_test_reduced = reduce_to_k_dim(M_test, k=2)

# Test proper dimensions
assert (M_test_reduced.shape[0] == 10), "M_reduced has {} rows; should have {}".format(M_test_reduced.shape[0], 10)
assert (M_test_reduced.shape[1] == 2), "M_reduced has {} columns; should have {}".format(M_test_reduced.shape[1], 2)

# Print Success
print ("-" * 80)
print("Passed All Tests!")
print ("-" * 80)
```

```
Running Truncated SVD over 10 words...
Done.
--------------------------------------------------------------------------------
Passed All Tests!
--------------------------------------------------------------------------------
```

Implement `plot_embeddings`

Here you will write a function to plot a set of 2D vectors in 2D space. For graphs, we will use Matplotlib (`plt`).

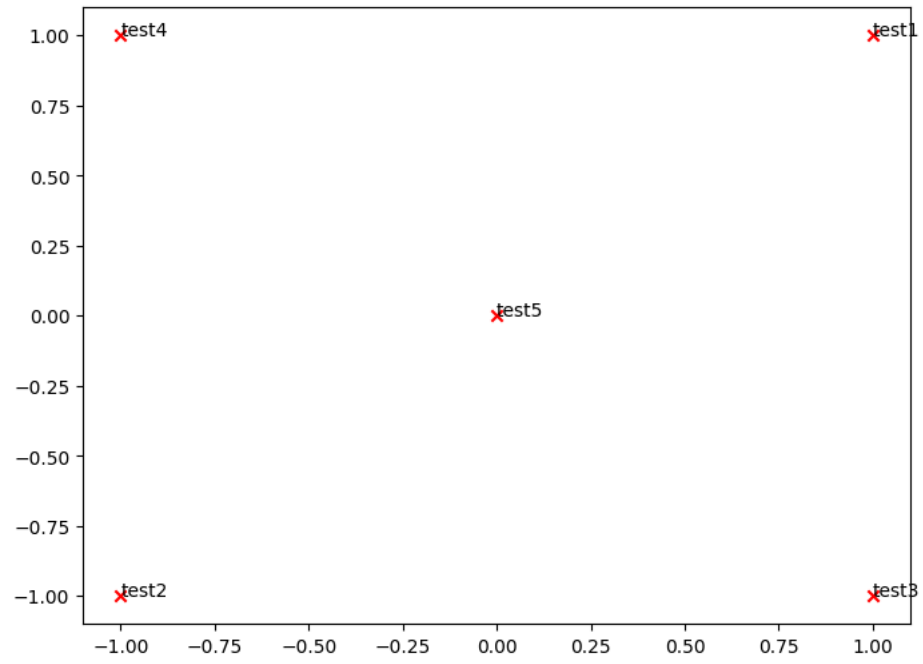
For this example, you may find it useful to adapt this code. In the future, a good way to make a plot is to look at the Matplotlib gallery, find a plot that looks somewhat like what you want, and adapt the code they give.

```
def plot_embeddings(M_reduced, word2ind, words):
    """ Plot in a scatterplot the embeddings of the words specified in the list "words".
        NOTE: do not plot all the words listed in M_reduced / word2ind.
        Include a label next to each point.

        Params:
            M_reduced (numpy matrix of shape (number of unique words in the corpus , 2)): matrix of 2-dimensioal word e
            word2ind (dict): dictionary that maps word to indices for matrix M
            words (list of strings): words whose embeddings we want to visualize
    """

    # ------------------
    # Write your implementation here.
    plt.figure(figsize=(8,6))
    for each_word in words:  # looping through words
        plt.scatter(M_reduced[word2ind[each_word],0], M_reduced[word2ind[each_word], 1], color='red', marker='x')
        plt.annotate(each_word, xy=(M_reduced[word2ind[each_word], 0], M_reduced[word2ind[each_word], 1]))

    # ------------------
```

```
# ---------------------
# Run this sanity check
# Note that this is not an exhaustive check for correctness.
# The plot produced should look like the included file question_1.4_test.png
# ---------------------

print ("-" * 80)
```

```
print ("Outputted Plot:")

M_reduced_plot_test = np.array([[1, 1], [-1, -1], [1, -1], [-1, 1], [0, 0]])
word2ind_plot_test = {'test1': 0, 'test2': 1, 'test3': 2, 'test4': 3, 'test5': 4}
words = ['test1', 'test2', 'test3', 'test4', 'test5']
plot_embeddings(M_reduced_plot_test, word2ind_plot_test, words)

print ("-" * 80)
```

```
--------------------------------------------------------------------------------
Outputted Plot:
--------------------------------------------------------------------------------
```



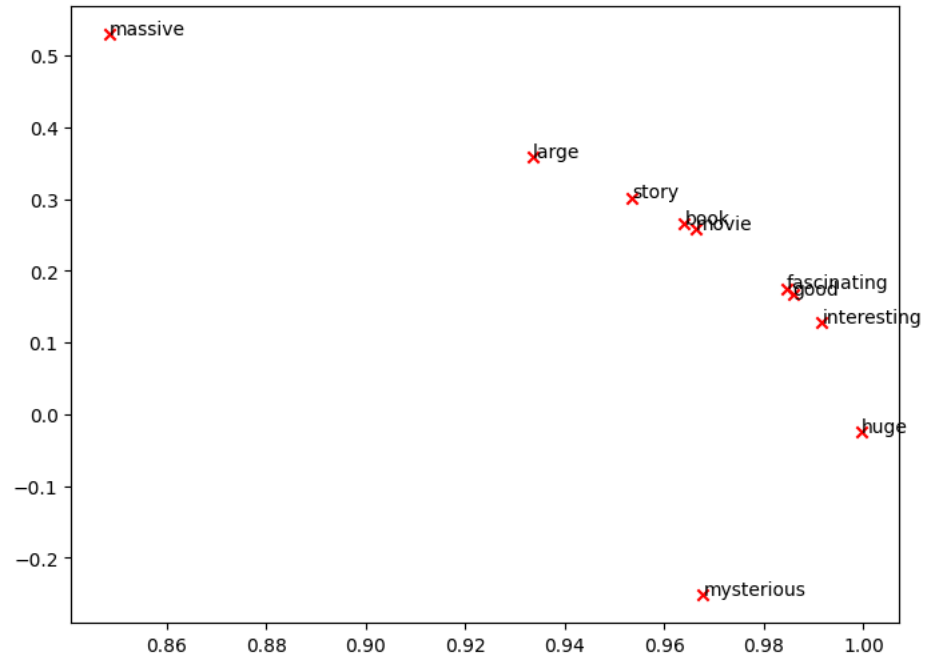
Co-Occurrence Plot Analysis

Now we will put together all the parts you have written! We will compute the co-occurrence matrix with fixed window of 4 (the default window size), over the Large Movie Review corpus. Then we will use TruncatedSVD to compute 2-dimensional embeddings of each word. TruncatedSVD returns U*S, so we need to normalize the returned vectors, so that all the vectors will appear around the unit circle (therefore closeness is directional closeness). **Note**: The line of code below that does the normalizing uses the NumPy concept of *broadcasting*. If you don't know about broadcasting, check out Computation on Arrays: Broadcasting by Jake VanderPlas.

Run the below cell to produce the plot. It can take up to a few minutes to run.

```
# ----------------------------
# Run This Cell to Produce Your Plot
# ----------------------------
imdb_corpus = read_corpus()
M_co_occurrence, word2ind_co_occurrence = compute_co_occurrence_matrix(imdb_corpus)
M_reduced_co_occurrence = reduce_to_k_dim(M_co_occurrence, k=2)

# Rescale (normalize) the rows to make them each of unit-length
M_lengths = np.linalg.norm(M_reduced_co_occurrence, axis=1)
M_normalized = M_reduced_co_occurrence / M_lengths[:, np.newaxis] # broadcasting

words = ['movie', 'book', 'mysterious', 'story', 'fascinating', 'good', 'interesting', 'large', 'massive', 'huge']

plot_embeddings(M_normalized, word2ind_co_occurrence, words)
```

```
Running Truncated SVD over 5880 words...
Done.
```



Verify that your figure matches "question_1.5.png" in the assignment zip. If not, use the figure in "question_1.5.png" to answer the next two questions.

a. Find at least two groups of words that cluster together in 2-dimensional embedding space. Give an explanation for each cluster you observe.

One group that was near each other was 'good', 'interesting' and 'fascinating'. These are all words people use to describe things, and they are kind of related. Another group that seemed pretty close was 'movie', 'book' and 'story' because they are all things we read or watch, like part of entertainment. So that they appeared near each other.

b. What doesn't cluster together that you might think should have? Describe at least two examples.

I thought 'massive', 'huge', and 'large' would be closer together because they all kind of mean the same thing like something really big. But 'massive' was a bit far off from the others. Even 'mysterious' is far. I expect it to be closer to 'book' since we usually describe a book as mysterious. It felt like it should have been part of that group or even bit closer.

Part 2: Prediction-Based Word Vector

We explore the embeddings produced by GloVe. Although GloVe is not purely 'prediction-based' like word2vec, its training objective is predictive. If you're feeling adventurous, challenge yourself and try reading GloVe's original paper .

Run the following cells to load the GloVe vectors into memory. **Note**: If this is your first time to run these cells, i.e. download the embedding model, it will take a couple minutes to run. If you've run these cells before, rerunning them will load the model without redownloading it, which will take about 1 to 2 minutes.

```
def load_embedding_model():
    """ Load GloVe Vectors
        Return:
            wv_from_bin: All 400000 embeddings, each length 200
    """
    import gensim.downloader as api
    wv_from_bin = api.load("glove-wiki-gigaword-200")
    print("Loaded vocab size %i" % len(list(wv_from_bin.index_to_key)))
    return wv_from_bin

# Load the GloVe embeddings
wv_from_bin = load_embedding_model()
```

```
Loaded vocab size 400000
```

Note: If you are receiving a "reset by peer" error, rerun the cell to restart the download.

Reducing dimensionality of Word Embeddings

Let's directly compare the GloVe embeddings to those of the co-occurrence matrix. In order to avoid running out of memory, we will work with a sample of 40000 GloVe vectors instead. Run the following cells to:

1. Put 40000 Glove vectors into a matrix M
2. Run `reduce_to_k_dim` (your Truncated SVD function) to reduce the vectors from 200-dimensional to 2-dimensional.

```
def get_matrix_of_vectors(wv_from_bin, required_words):
    """ Put the GloVe vectors into a matrix M.
        Param:
            wv_from_bin: KeyedVectors object; the 400000 GloVe vectors loaded from file
        Return:
            M: numpy matrix shape (num words, 200) containing the vectors
            word2ind: dictionary mapping each word to its row number in M
    """
    import random
    words = list(wv_from_bin.index_to_key)
    print("Shuffling words ...")
    random.seed(225)
    random.shuffle(words)
    print("Putting %i words into word2ind and matrix M..." % len(words))
    word2ind = {}
    M = []
    curInd = 0
    for w in words:
        try:
            M.append(wv_from_bin.get_vector(w))
            word2ind[w] = curInd
            curInd += 1
        except KeyError:
            continue
    for w in required_words:
        if w in words:
            continue
        try:
            M.append(wv_from_bin.get_vector(w))
            word2ind[w] = curInd
            curInd += 1
        except KeyError:
            continue
    M = np.stack(M)
    print("Done.")
    return M, word2ind
```

```
# -----------------------------------------------------------------
# Run Cell to Reduce 200-Dimensional Word Embeddings to k Dimensions
# Note: This should be quick to run
# -----------------------------------------------------------------
M, word2ind = get_matrix_of_vectors(wv_from_bin, words)
M_reduced = reduce_to_k_dim(M, k=2)

# Rescale (normalize) the rows to make them each of unit-length
M_lengths = np.linalg.norm(M_reduced, axis=1)
M_reduced_normalized = M_reduced / M_lengths[:, np.newaxis] # broadcasting
```

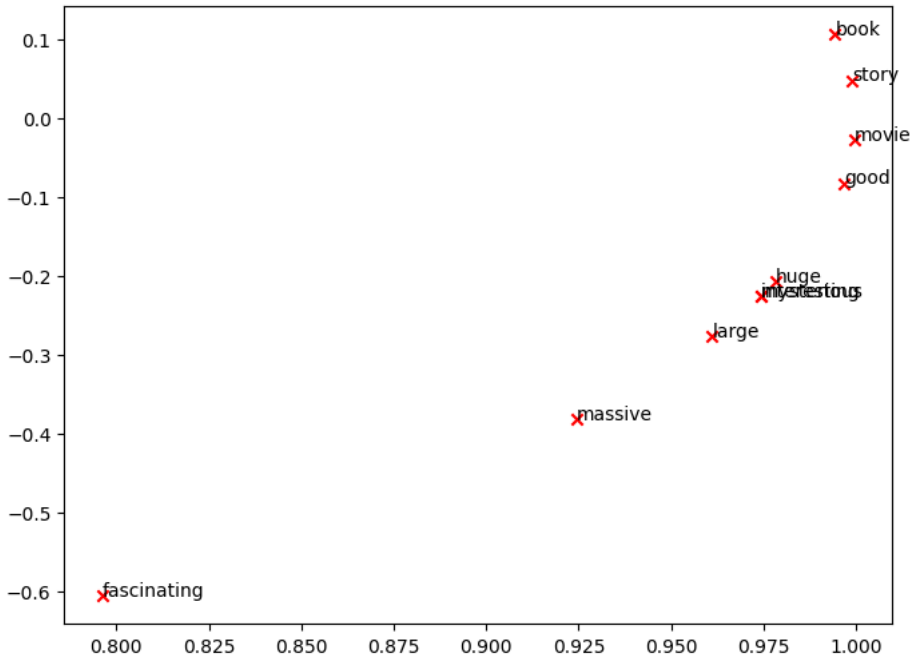
```
Shuffling words ...
Putting 400000 words into word2ind and matrix M...
Done.
Running Truncated SVD over 400000 words...
Done.
```

Note: If you are receiving out of memory issues on your local machine, try closing other applications to free more memory on your device. You may want to try restarting your machine so that you can free up extra memory. Then immediately run the jupyter notebook and see if you can load the word vectors properly. If you still have problems with loading the embeddings onto your local machine after this, please go to office hours or contact course staff.

GloVe Plot Analysis

Run the cell below to plot the 2D GloVe embeddings for `['movie', 'book', 'mysterious', 'story', 'fascinating', 'good', 'interesting', 'large', 'massive', 'huge']`.

```
words = ['movie', 'book', 'mysterious', 'story', 'fascinating', 'good', 'interesting', 'large', 'massive', 'huge']

plot_embeddings(M_reduced_normalized, word2ind, words)
```



Verify that your figure matches "question_2.1.png" in the assignment zip. If not, use the figure in "question_2.1.png" (and the figure in "question_1.5.png", if applicable) to answer the next two questions.

a. What is one way the plot is different from the one generated earlier from the co-occurrence matrix? What is one way it's similar?

One big difference is that in question_2.1.png plot, some words like ‘fascinating’ are far away from everything else, while in the co-occurrence plot earlier, most words were grouped more closely. But one thing that is kind of similar is that ‘movie’, ‘book’, and ‘story’ are still near each other in both plots, which makes sense because they are all related to entertainment or storytelling.

b. Why might the GloVe plot (question_2.1.png) differ from the plot generated earlier from the co-occurrence matrix (question_1.5.png)?

The question_2.1.png plot is bit different because it was trained in a different way. Instead of just counting nearby words like the co-occurrence matrix does, GloVe also looks at how often words appear together across the whole dataset. So the relationships it learns are more global, and that changes where the words show up in the plot.

Cosine Similarity

Now that we have word vectors, we need a way to quantify the similarity between individual words, according to these vectors. One such metric is cosine-similarity. We will be using this to find words that are "close" and "far" from one another.

We can think of n-dimensional vectors as points in n-dimensional space. If we take this perspective L1 and L2 Distances help quantify the amount of space "we must travel" to get between these two points. Another approach is to examine the angle between two vectors. From trigonometry we know that:

Instead of computing the actual angle, we can leave the similarity in terms of $similarity = cos(\Theta)$. Formally the Cosine Similarity s between two vectors p and q is defined as:

$$s = \frac{p \cdot q}{||p|| ||q||}, \text{ where } s \in [-1, 1]$$

Polysemes and homonyms are words that have more than one meaning (see this wiki page to learn more about the difference between polysemes and homonyms). Find a word with *at least two VERY different meanings* such that the top-10 most similar words (according to cosine similarity) contain related words from *both* meanings. For example, a noun **"bat"** has both "a flying mammal" and "a sports equipment" meanings in the top 10, and a verb **"strike"** has both "to hit" and "to protest" meanings. You will probably need to try several polysemous or homonymic words before you find one.

Give one noun and one verb (not "bat" or "strike") that are polysemous and have VERY different meanings in the top 10 most similar words. Your score will depend on your choice of the words and the **explanations** you write in the comment section below.

Note: You should use the `wv_from_bin.most_similar(word)` function to get the top 10 most similar words of a given word. This function ranks all other words in the vocabulary with respect to their cosine similarity to the given word.

```
# ------------------
# Write your implementation here.

print("light") # noun
print(wv_from_bin.most_similar("light"))

print("pass") #verb
print(wv_from_bin.most_similar("pass"))
# ------------------
```

```
light
[('bright', 0.6242774724960327), ('dark', 0.6141002178192139), ('lights', 0.6013951897621155), ('lighter', 0.5581752657
pass
[('passes', 0.772192656993866), ('passing', 0.7188892960548401), ('passed', 0.6851356625556946), ('kick', 0.621737360954
```

The word light is polysemous. Some top most similar words like “sunlight”, “bright”, and “lights” show its meaning as illumination. Others like “lighter” and “heavy” show its meaning as weight. These are two very different meanings, and both are reflected in the similar word. "Pass" is a verb with multiple meanings. The similar words include “kick”, “touchdown”, and “ball”, which suggest the sports meaning passing the ball. At the same time, “passed”, “passing”, and “passes” hint at success or movement. This shows that the word has very different uses.

When considering Cosine Similarity, it's often more convenient to think of Cosine Distance, which is simply 1 - Cosine Similarity.

Find three words (w_1, w_2, w_3) where w_1 and w_2 are synonyms and w_1 and w_3 are antonyms, but Cosine Distance $(w_1, w_3) <$ Cosine Distance (w_1, w_2).

As an example, w_1="happy" is closer to w_3="sad" than to w_2="cheerful". Please find a different example that satisfies the above. Once you have found your example, please give a possible explanation for why this counter-intuitive result may have happened.

Do NOT use the example above ("happy", "sad", "cheerful") in your answer/implementation. **Think of examples by yourself.**

You should use the the `wv_from_bin.distance(w1, w2)` function here in order to compute the cosine distance between two words. Please see the **GenSim documentation** for further assistance.

```
# ------------------
# Write your implementation here.
w1 = "hot"
w2 = "warm"    # w1 synonym
w3 = "cold"    # w1 antonym
print("Distance (w1(hot), w2(warm)):", wv_from_bin.distance(w1, w2))
print("Distance (w1(hot), w3(cold)):", wv_from_bin.distance(w1, w3))
# ------------------
```

```
Distance (w1(hot), w2(warm)): 0.4111672639846802
Distance (w1(hot), w3(cold)): 0.40621882677078247
```

I used “hot” as w1. Its synonym is “warm” and its antonym is “cold”. The distance between hot(w1) and cold(w3) (0.4062) is less than the distance between hot(w1) and warm(w2) (0.4111), even though warm is more similar in meaning than cold. This happens because we use “hot” and “cold” more often in everyday conversations than hot and warm. Since word vectors learn from context, “hot” and “cold” end up being close in vector space even though their meanings are opposite. That is why cosine distance shows them as more similar.

Word vectors have been shown to *sometimes* exhibit the ability to solve analogies.

As an example, for the analogy "man : grandfather :: woman : x" (read: man is to grandfather as woman is to x), what is x?

In the cell below, we show you how to use word vectors to find x using the `most_similar` function from the **GenSim documentation**. The function finds words that are most similar to the words in the `positive` list and most dissimilar from the words in the `negative` list (while omitting the input words, which are often the most similar; see this paper). The answer to the analogy will have the highest cosine similarity (largest returned numerical value).

```
# Run this cell to answer the analogy -- man : grandfather :: woman : x
pprint.pprint(wv_from_bin.most_similar(positive=['woman', 'grandfather'], negative=['man']))
```

```
[('grandmother', 0.7608445286750793),
 ('granddaughter', 0.7200808525085449),
 ('daughter', 0.7168302536010742),
 ('mother', 0.7151536345481873),
 ('niece', 0.7005682587623596),
 ('father', 0.6659887433052063),
 ('aunt', 0.6623408794403076),
 ('grandson', 0.6618767976760864),
 ('grandparents', 0.644661009311676),
 ('wife', 0.6445354223251343)]
```

Let m, g, w, and x denote the word vectors for `man`, `grandfather`, `woman`, and the answer, respectively. Using **only** vectors m, g, w, and the vector arithmetic operators $+$ and $-$ in your answer, what is the expression in which we are maximizing cosine similarity with x?

Hint: Recall that word vectors are simply multi-dimensional vectors that represent a word. It might help to draw out a 2D example using arbitrary locations of each vector. Where would `man` and `woman` lie in the coordinate plane relative to `grandfather` and the answer?

"man : grandfather :: woman : x", we want to find a vector x such that x ~ (g - m + w) is maximum. This is basically starting from grandfather, we subtract the man part and add woman. So we're shifting the male gender aspect to female while keeping the grandparent meaning. We are maximizing the cosine similarity between x and all other word vectors to find the best match, which ideally would be grandmother.

a. For the previous example, it's clear that "grandmother" completes the analogy. But give an intuitive explanation as to why the `most_similar` function gives us words like "granddaughter", "daughter", or "mother?

Even though grandmother is the best match, other words like granddaughter, daughter or mother also appear because they are close in meaning as they are all females, frequently appear in similar contexts like family conversations, and in the vector space they are located near each other. Word vectors do not just look for perfect one to one meanings. They group related ideas together, so all these words feel similar in the way people use them in language.

b. Find an example of analogy that holds according to these vectors (i.e. the intended word is ranked top). In your solution please state the full analogy in the form x:y :: a:b. If you believe the analogy is complicated, explain why the analogy holds in one or two sentences.

Note: You may have to try many analogies to find one that works!

```
# For example: x, y, a, b = ("", "", "", "")
# ----------------
# Write your implementation here.

x, y, a, b = ("morning", "breakfast", "evening", "dinner")
```

```
# ------------------

# Test the solution
assert wv_from_bin.most_similar(positive=[a, y], negative=[x])[0][0] == b
```

morning:breakfast::evening:dinner. This analogy works because both “morning” and “evening” are times of the day, and “breakfast” and “dinner” are the meals typically associated with those times. So, just like breakfast is something people usually have in the morning, dinner is something they usually have in the evening. That connection between time and meal makes this analogy strong.

a. Below, we expect to see the intended analogy "hand : glove :: foot : **sock**", but we see an unexpected result instead. Give a potential reason as to why this particular analogy turned out the way it did?

```
pprint.pprint(wv_from_bin.most_similar(positive=['foot', 'glove'], negative=['hand']))
```

```
[('45,000-square', 0.4922032654285431),
 ('15,000-square', 0.4649604558944702),
 ('10,000-square', 0.4544755816459656),
 ('6,000-square', 0.44975775480270386),
 ('3,500-square', 0.444133460521698),
 ('700-square', 0.44257497787475586),
 ('50,000-square', 0.4356396794319153),
 ('3,000-square', 0.43486514687538147),
 ('30,000-square', 0.4330596923828125),
 ('footed', 0.43236875534057617)]
```

"hand:glove::foot:sock" did not return "sock" because the model did not learn strong connections between "foot" and "sock" the way it did for "hand" and "glove". It might be that in the training data, "glove" and "hand" appeared together more often in meaningful ways, while "sock" might not have had enough similar usage patterns with "foot". Instead, the word "foot" have appeared frequently in phrases like "45000-square-foot", which is why the model returned results like "45,000-square" it is simply capturing what it saw most in the text it was trained on.

b. Find another example of analogy that does *not* hold according to these vectors. In your solution, state the intended analogy in the form x:y :: a:b, and state the **incorrect** value of b according to the word vectors (in the previous example, this would be **'45,000-square'**).

```
# For example: x, y, a, b = ("", "", "", "")
# ------------------
# Write your implementation here.

x, y, a, b = ("bird", "wing", "fish", "fin")

# ------------------
pprint.pprint(wv_from_bin.most_similar(positive=[a, y], negative=[x]))
assert wv_from_bin.most_similar(positive=[a, y], negative=[x])[0][0] != b
```

```
[('wings', 0.42199045419692993),
 ('serving', 0.41504913568496704),
 ('serve', 0.40171900391578674),
 ('front', 0.3889312148094177),
 ('rightwing', 0.3833450376987457),
 ('right', 0.3824288845062256),
 ('nationalist', 0.3817332684993744),
 ('fuselage', 0.3777334690093994),
 ('liberal', 0.3770541846752167),
 ('served', 0.37605786323547363)]
```

This failed because the word "wing" appeared in many contexts unrelated to birds like political "rightwing", airplane wings, or expressions like "under one's wing." These dominant contexts distort the embedding space causing the model to return "wings" and political terms instead of the more biologically accurate "fin." It reflects how word vectors are influenced by frequency and context in text, not by real world logic or anatomy.

Guided Analysis of Bias in Word Vectors

It's important to be cognizant of the biases (gender, race, sexual orientation etc.) implicit in our word embeddings. Bias can be dangerous because it can reinforce stereotypes through applications that employ these models.

Run the cell below, to examine (a) which terms are most similar to "man" and "profession" and most dissimilar to "woman" and (b) which terms are most similar to "woman" and "profession" and most dissimilar to "man". Point out the difference between the list of female-associated words and the list of male-associated words, and explain how it is reflecting gender bias.

```
# Run this cell
# Here `positive` indicates the list of words to be similar to and `negative` indicates the list of words to be
# most dissimilar from.

pprint.pprint(wv_from_bin.most_similar(positive=['man', 'profession'], negative=['woman']))
print()
pprint.pprint(wv_from_bin.most_similar(positive=['woman', 'profession'], negative=['man']))
```

```
[('reputation', 0.5250176787376404),
 ('professions', 0.5178037881851196),
 ('skill', 0.49046966433525085),
 ('skills', 0.49005505442619324),
 ('ethic', 0.4897659420967102),
 ('business', 0.4875852167606354),
 ('respected', 0.485920250415802),
 ('practice', 0.482104629278183),
 ('regarded', 0.4778572618961334),
 ('life', 0.4760662019252777)]

[('professions', 0.5957457423210144),
 ('practitioner', 0.49884122610092163),
 ('teaching', 0.48292139172554016),
 ('nursing', 0.48211804032325745),
 ('vocation', 0.4788965880870819),
 ('teacher', 0.47160351276397705),
 ('practicing', 0.46937814354896545),
 ('educator', 0.46524327993392944),
 ('physicians', 0.4628995358943939),
 ('professionals', 0.4601394236087799)]
```

When we check the similarity for ['man', 'profession'] versus ['woman'], the results include words like reputation, skill, business, and ethic. These words seem to reflect stereotypical traits often associated with men in professional roles like being career-driven or focused on reputation and skill. But when we do ['woman', 'profession'] versus ['man'], the results include words like teaching, nursing, teacher, and educator which are traditionally considered more nurturing or caregiving professions often linked to women. This shows that the word vectors are capturing gender stereotypes from the data they were trained on, reinforcing old biases about what kinds of jobs men and women do.

Use the `most_similar` function to find another pair of analogies that demonstrates some bias is exhibited by the vectors. Please briefly explain the example of bias that you discover.

```
# ------------------
# Write your implementation here.

pprint.pprint(wv_from_bin.most_similar(positive=['doctor', 'woman'], negative=['nurse']))
# ------------------
```

```
[('man', 0.6883388161659241),
 ('person', 0.639634370803833),
 ('he', 0.5910089612007141),
 ('she', 0.5881379842758179),
 ('her', 0.5810558795928955),
 ('him', 0.575454592704773),
 ('another', 0.5702431797981262),
 ('himself', 0.5684657096862793),
 ('someone', 0.564157247543335),
 ('who', 0.5635774731636047)]
```

I tried nurse:woman::doctor:x, and ran the expression.The top result was "man", which shows a clear gender bias in the word vectors. It reflects the stereotype that doctors are more male-associated and nurses are more female-associated. Even though both are healthcare professions, the model reinforces biased societal associations between gender and

a. Give one possible explanation of how bias gets into the word vectors. Your explanation should be focused on word vectors, as opposed to bias in other AI systems (e.g., ChatGPT). You can use specific historical examples to back up your explanations if necessary.

Bias enters word vectors because they are trained on large text collected from the internet, books, news articles, and other real-world sources. These sources naturally reflect human biases for example, associating "nurse" more with "woman" and "engineer" with "man" and the word embeddings learn those patterns. Since word vectors capture statistical co-occurrence, they encode societal stereotypes if such pairings are frequent in the data. A well-known example is man:computer programmer::woman: homemaker, which reflects historical bias in data, not actual capability.

b. What is one possible method you can use to mitigate bias exhibited by word vectors? Briefly explain the method and what the goal of the method was.

Word embeddings can reflect human biases because they are trained on real-world data like news articles and books. So if those sources have stereotypes, the embeddings will too. For example, it might connect words like “man” with “programmer” and “woman” with “homemaker,” even though that is not always true. One way to deal with this is by adjusting the embeddings so that gendered words like “he” and “she” do not influence neutral words like “doctor” or “leader.” They try to remove the part of the vector that carries the bias, so the model becomes more fair.